

DOCUMENT NO. SRS-02 · REV 0 (DRAFT)

Software Requirement Specification

Neural Network from Scratch in C (Educational)
An MLP written in plain C, designed to be *read*

Project	neural_net_c — from-scratch multilayer perceptron in C99
Document ID	SRS-02
Date	2026-07-13
Revision	0 — Draft for approval
Author	Tuğrul Altınüzengi (with Claude)
Source repository	D:\Dekstop\Coding\n neural_net_c\

Contents

1. Introduction — purpose, scope, building-block checklist
2. Deliverables & Project Structure
3. Functional Requirements (NN)
4. Non-Functional (Readability) Requirements (NR)
5. Acceptance Criteria
6. Risks

1 Introduction

1.1 Purpose

Produce a C program whose **primary output is understanding**: every fundamental building block of a feed-forward neural network — neuron, weight, bias, layer, activation function, forward pass, loss, gradient, backpropagation, training loop — must be visible as a small, named, commented piece of C code that maps 1:1 onto the underlying math. Accuracy numbers are secondary; **readability and traceability are the product**.

1.2 Scope — progressive, one codebase

- **Stage 1 — XOR.** A tiny MLP (2 inputs → 4 hidden → 1 output) learns XOR. Small enough that a single training step can be traced by hand on paper against the program's own printed trace.
- **Stage 2 — MNIST.** The **same library code**, no rewrites, scaled up (784 → 128 → 10) to classify handwritten digits from the real MNIST dataset. Proves the building blocks generalize.

Out of scope: GPU, threading, convolutions, autograd frameworks, any external library beyond libc + `math.h`. PC only (Windows, gcc/MinGW); no ESP32 port in this project.

1.3 The Building Blocks the Code Must Expose (checklist)

Each item below must exist as its **own clearly named function or struct**, with a comment stating the math it implements:

1. **Neuron / Layer** — `Layer` struct: weight matrix `W`, bias vector `b`, and the per-forward-pass caches `z` (pre-activation) and `a` (activation).
2. **Weighted sum** — $z = W \cdot x + b$ (explicit loops, no matrix library).
3. **Activation functions** — `sigmoid`, `relu`, `softmax`, each defined next to its derivative (`*_prime`).
4. **Forward pass** — `nn_forward()`: input → layer by layer → output.
5. **Loss** — MSE (Stage 1) and cross-entropy (Stage 2), each with the derivative used to start backpropagation.
6. **Backpropagation** — `nn_backward()`: the chain rule written out — output delta, hidden delta $\delta^l = (W^{l+1 T} \delta^{l+1}) \odot \sigma'(z^l)$, and the gradients $\partial L / \partial W = \delta \cdot a^T$, $\partial L / \partial b = \delta$.
7. **Parameter update** — plain SGD $w \leftarrow w - \eta \cdot \partial L / \partial w$ (optionally mini-batch averaging in Stage 2).

8. **Training loop** — epochs, shuffling, periodic loss/accuracy reporting.

9. **Initialization** — seeded PRNG, small random weights (Xavier-style scaling, explained in a comment).

2 Deliverables & Project Structure

```
neural_net_c/
├── SRS_NEURAL_NETWORK_C.md    (this document, markdown master)
├── EXPLANATION.md            the "textbook": math → code walkthrough
├── Makefile                  gcc builds; also documents the plain commands
├── src/
│   ├── nn.h / nn.c          Layer + Network structs, forward, backward, update
│   ├── activations.h / .c    sigmoid / relu / softmax + derivatives
│   ├── loss.h / .c           mse / cross-entropy + derivatives
│   ├── data_mnist.h / .c     IDX file loader (magic 0x00000803 / 0x00000801)
│   ├── main_xor.c            Stage 1 entry point
│   ├── main_mnist.c          Stage 2 entry point
│   └── gradcheck.c           numerical-vs-analytical gradient verification
└── data/                    MNIST IDX files (download instructions in EXPLANATION.md)
```

3 Functional Requirements

ID	Requirement
NN-1	Builds with <code>make</code> (and each target also with a single documented <code>gcc</code> command line) under MinGW/gcc on Windows; C99; only libc + <code>-lm</code>
NN-2	<code>xor</code> binary: trains a 2-4-1 sigmoid MLP to MSE < 0.01, then prints the truth table (outputs ≈ 0, 1, 1, 0). Runtime < 1 s
NN-3	<code>xor --trace</code> : prints every number of one full training step (each z , a , δ , gradient, updated weight) so it can be checked by hand — the core learning tool
NN-4	<code>gradcheck</code> binary: compares analytical backprop gradients against centered finite differences ($L(w+\epsilon) - L(w-\epsilon) / 2\epsilon$); passes iff max relative error < $1e-4$. This <i>proves</i> the backprop code is correct
NN-5	<code>mnist</code> binary: loads the 4 IDX files, trains 784-128-10 (ReLU hidden, softmax output, cross-entropy, mini-batch SGD), reaches ≥ 95 % test accuracy within ≤ 10 epochs / ≤ 5 min on the PC
NN-6	Per-epoch reporting: epoch, training loss, test accuracy, elapsed seconds
NN-7	<code>--seed N</code> makes runs bit-for-bit reproducible; default seed fixed
NN-8	<code>mnist --save w.bin / --load w.bin --eval</code> : save/load weights, inference-only mode; <code>--predict <index></code> prints one test image as ASCII art with predicted vs true label
NN-9	Missing/corrupt data files → clear error message with download instructions; no crash

4 Non-Functional (Readability) Requirements

ID	Requirement
NR-1	No function > ~40 lines; no file > ~300 lines

NR-2	Variable names match the math notation used in EXPLANATION.md (<code>w</code> , <code>b</code> , <code>z</code> , <code>a</code> , <code>delta</code> , <code>eta</code>)
NR-3	Every non-trivial loop has a comment saying which formula it computes
NR-4	Explicit <code>for</code> -loops over neurons — no clever pointer tricks, no macro magic
NR-5	Memory: allocated once at network creation, freed once at destruction; sizes visible in the struct
NR-6	Console output ASCII-only (Turkish Windows cp1254 console)
NR-7	EXPLANATION.md walks: perceptron → why hidden layers (XOR!) → chain rule → backprop derivation → mini-batches; each section ends with “this is function X in file Y, lines ...”

5 Acceptance Criteria

ID	Criterion
AC-1	<code>make && xor</code> prints a correct XOR truth table (NN-2)
AC-2	A hand-calculation of one step from <code>xor --trace</code> matches the printout (NN-3) — done together as a learning session
AC-3	<code>gradcheck</code> passes (NN-4)
AC-4	<code>mnist</code> reaches ≥ 95 % test accuracy, reproducibly with the default seed (NN-5, NN-7)
AC-5	<code>mnist --load w.bin --predict 7</code> shows the digit as ASCII art with the correct label (NN-8)
AC-6	Reading EXPLANATION.md side-by-side with the code, every §1.3 building block can be pointed at within 10 seconds

6 Risks

Risk	Mitigation
MNIST download links rot (yann.lecun.com is flaky)	EXPLANATION.md lists mirrors (e.g. GitHub-hosted copies); loader also accepts a documented CSV fallback
95 % target vs simplicity tension	784-128-10 + ReLU + mini-batch SGD reliably reaches ~97 %; no tricks needed
Educational trace floods the console on MNIST	<code>--trace</code> is XOR-only by design